

Days 9-12

Online softmax, FlashAttention, flash-decoding, and continuous batching. Continues the Days 1-4 and Days 5-8 notes. Same model and card: Qwen2.5-1.5B, fp16, one RTX 3060 12GB.

Day	Topic	The one thing you must be able to derive
9	Online softmax	The running-max rescale recurrence, cold, on a blank page
10	FlashAttention	Why it is an IO win and not a FLOPs win
11	Flash-decoding	Why FlashAttention does nothing at batch 1, and what does
12	Continuous batching	Your two measured plateaus, and which ceiling caused each

Why these four days carry more weight than any others. FlashAttention is the single most likely deep-dive question in a systems interview, and it is the one anchor answer this repo **cannot** support with a measurement. The derivation has to carry it. So Day 9 is not background for Day 10, it *is* the answer: if you can write the recurrence unaided, the rest of FlashAttention is a paragraph of consequences.

Day 9 · Online softmax: computing a softmax you cannot see all of

9.1 The problem

Softmax needs a denominator that sums over the whole row. Written naively that means you must hold the entire row of scores before you can normalise any of it. For a 4096-token sequence that row is 4096 numbers per query, per head, and the full score matrix does not fit in on-chip memory. So the standard implementation writes it to VRAM and reads it back.

The question this day answers: **can you produce the exact softmax-weighted output while only ever seeing one tile of the row at a time?** The answer is yes, exactly, with no approximation. That fact is what FlashAttention is built on.

9.2 The recurrence

Recall the max-subtraction trick from Day 1: subtracting a constant from every score leaves softmax unchanged. Online softmax exploits that in the other direction. Process tiles in order, keeping a *running* maximum. When a later tile contains a bigger maximum, every quantity accumulated so far was normalised against the wrong constant, so you rescale it by a single correction factor and carry on.

Carry three running values: `m` the max seen so far, `l` the sum of exponentials, and `o` the accumulated weighted values. For a new tile of scores `s` with values `v`:

```
m_new = max(m_old, max(s))
c      = exp(m_old - m_new)           the correction factor, always in (0, 1]

l_new = l_old * c + sum_j exp(s_j - m_new)
o_new = o_old * c + sum_j exp(s_j - m_new) * v_j

at the end: output = o_final / l_final
```

That is the whole thing. Two accumulators get multiplied by the same scalar `c` whenever the max moves. Everything else is the ordinary softmax you already know.

EXAMPLE 9-A: THE RECURRENCE, WORKED, AND CHECKED AGAINST THE DIRECT ANSWER

```
scores s = [1, 3, 2, 5]   values v = [10, 20, 30, 40]   processed as two tiles

TILE 1: s = [1, 3]
  m = 3
  exp(1-3) = 0.135335      exp(3-3) = 1.000000
  l = 0.135335 + 1.000000      = 1.135335
  O = 0.135335*10 + 1.000000*20      = 21.353350

TILE 2: s = [2, 5]      max of tile is 5, which is bigger than m = 3
  m_new = 5
  c      = exp(3 - 5) = 0.135335      <- rescale everything so far
  exp(2-5) = 0.049787      exp(5-5) = 1.000000
  l = 1.135335*0.135335 + (0.049787 + 1.000000) = 1.203438
  O = 21.353350*0.135335 + (0.049787*30 + 40) = 44.383460

  output = 44.383460 / 1.203438 = 36.8812

DIRECT CHECK, all four at once with global max 5:
weights = [exp(-4), exp(-2), exp(-3), exp(0)]
         = [0.018316, 0.135335, 0.049787, 1.000000]   sum = 1.203438   MATCHES l
weighted = 0.18316 + 2.70670 + 1.49361 + 40.0 = 44.383470   MATCHES O
output   = 36.8812                                           IDENTICAL
```

Say this out loud. The two results are identical, not close. Online softmax is exact. It is not a numerical approximation, an estimate, or a trade of accuracy for speed. Every FlashAttention output is bit-comparable to the standard implementation up to floating-point associativity. Interviewers probe this because a lot of people assume there must be a quality cost.

9.3 Why the correction factor is always safe

$c = \exp(m_{\text{old}} - m_{\text{new}})$ and $m_{\text{new}} \geq m_{\text{old}}$ by construction, so the exponent is zero or negative and c lands in $(0, 1]$. It can never overflow. Every $\exp(s_j - m_{\text{new}})$ is likewise bounded above by 1. The recurrence is numerically safer than the naive version, not riskier, which is worth saying because it sounds like it should be the opposite.

Sources: [Milakov & Gimelshein, Online normalizer calculation for softmax](#) · [Dao et al., FlashAttention](#) (Algorithm 1 is this recurrence)

Day 10 · FlashAttention: the same FLOPs, far fewer bytes

10.1 What the standard implementation actually does to memory

Standard attention is four steps, and three of them round-trip through VRAM:

$S = Q K^T$	write S to HBM	$[N, N]$
$P = \text{softmax}(S)$	read S , write P	$[N, N]$
$O = P V$	read P	$[N, d]$

The N -by- N score matrix is never wanted afterwards. It exists only to be consumed by the next operation. But it is too large for on-chip memory, so it is written out and read back, twice.

EXAMPLE 10-A: THE COST OF THAT ROUND TRIP, ON YOUR OWN CONFIG

prefill, 512 tokens, 12 heads, head_dim 128, fp16, ONE layer

score matrix per head	512 x 512 x 2 B	=	512 KiB
x 12 heads		=	6 MiB
traffic: write S , read S , write P , read P		=	24 MiB

the actual inputs, for comparison:

Q, K, V each 512 x 1536 x 2 B	=	1.5 MiB, three	=	4.5 MiB
---------------------------------	---	----------------	---	---------

so 24 MiB of traffic on a throwaway intermediate against 4.5 MiB of real input,
a 5.3x overhead, per layer, x 28 layers = 672 MiB per prefill

and it is QUADRATIC. at 4096 tokens, same model:

score matrix per layer	4096 x 4096 x 2 x 12	=	384 MiB
traffic per layer		=	1.5 GiB
x 28 layers		=	43 GiB
time at 291.5 GB/s		=	148 ms

148 ms of moving a matrix that gets thrown away.

10.2 What FlashAttention changes

It never materialises S . Q, K and V are cut into tiles small enough to fit in an SM's shared memory. For each tile of queries, it loops over tiles of keys and values, computes that small block of scores *inside SRAM*, and folds it into the running (m, l, O) using the Day 9 recurrence. The block of scores is discarded before the next one is computed. Only Q, K, V and the final O ever touch VRAM.

```
for each tile of Q:
    m, l, O = -inf, 0, 0
    for each tile of K, V:
        load K_tile, V_tile into SRAM
        s = Q_tile K_tile^T          <- lives and dies in SRAM
        apply the online softmax update to (m, l, O)
    write O_tile / l to HBM
```

The sentence that gets this right. FlashAttention does **not** reduce FLOPs. It performs the same $2 N^2 d$ multiply-accumulates for QK^T and the same again for PV , and in the backward pass it deliberately *recomputes* the scores rather than storing them, so it does strictly more arithmetic. It is faster because attention was never limited by arithmetic. It is IO-aware: it changes what crosses the HBM boundary. Anyone who says "FlashAttention makes attention $O(N)$ instead of $O(N^2)$ " has it wrong. The compute is still quadratic. The *memory* is linear.

10.3 Why the tiling is possible at all

Two properties have to hold together, and this is the connection an interviewer is looking for:

1. **Online softmax** (Day 9) means the row denominator does not have to be known in advance. This is the mathematical permission to tile.
2. **Multi-head attention** (Day 1) means each head's score block is small. With head_dim 128 and a block of 64 queries by 128 keys, the tile is a few tens of KiB, which fits in the up-to-128 KiB of shared memory per SM. This is the hardware permission to tile.

Remove either one and the technique collapses. That is why the Day 1 remark that "multi-head is free" has a second payoff here.

10.4 FlashAttention-2, in one paragraph

FA1 was already IO-optimal in the sense that mattered, so FA2 attacks occupancy instead. It reduces the number of non-matmul operations (which run far slower than tensor-core matmuls, so they cost more wall clock than their FLOP count suggests), it parallelises over the sequence-length dimension rather than only over batch and heads (so a single long sequence can fill the SMs), and it splits work between warps to cut shared-memory synchronisation. Result is roughly 2x over FA1 with no change to the mathematics. Note that all three of those are Day 3 concepts: occupancy, latency hiding, warp scheduling.

Sources: [Dao et al., FlashAttention](#) · [Dao, FlashAttention-2](#) · [flash-attention repo](#)

Day 11 • Flash-decoding: what to do when the query length is 1

11.1 FlashAttention does nothing for batch-1 decode. Two independent reasons.

This is the follow-up question after the FA derivation, and it separates people who read the paper from people who understood it.

EXAMPLE 11-A: THE DECODE SCORE MATRIX IS NOT A MATRIX

```
prefill, 512 tokens:  S is [512, 512] = 262,144 scores    <- the thing FA avoids writing
decode, one token:   S is [1, n]                        <- a single row

at n = 512:  1 x 512 x 2 B = 1 KiB.  It fits in registers.
```

1. **There is nothing to avoid materialising.** The intermediate FA exists to keep out of HBM is 1 KiB. There is no quadratic term, because the query dimension is 1. FA's entire saving is zero here.
2. **The parallelism disappears.** FA1 and FA2 assign work by splitting the query dimension across thread blocks. With one query there is one block per (head, batch). At batch 1 that is 12 blocks of work for **28 SMs**. More than half the GPU is idle by construction.

And the actual cost of decode attention is neither of those: it is reading the KV cache. At 512 context this model reads $512 \times 28 \text{ KiB} = 14 \text{ MiB}$ of cache per step, and no amount of tiling reduces that. You must read every key and value to attend to them.

11.2 The fix: split along the KV sequence instead

Flash-decoding, also called split-KV, changes which axis is partitioned. Cut the KV cache into chunks and give each chunk to a different SM. Each computes a *partial* attention output over its chunk and returns three things: its partial output, its running max m , and its running sum l . A second kernel then combines the partials.

The combine step is the Day 9 recurrence again, applied across chunks instead of across tiles:

```
m = max over chunks of m_i
0 = sum_i 0_i * exp(m_i - m)      l = sum_i l_i * exp(m_i - m)
output = 0 / l
```

The same rescale, one level up. That is why online softmax is worth deriving properly: it appears twice, once inside the kernel and once across kernels.

11.3 This is in your own trace

From `results/trace_vllm-graph.json.gz`, one steady-state decode step of Qwen2.5-1.5B on the 3060:

EXAMPLE 11-B: FLASH-DECODING, COUNTED IN YOUR TRACE

```
flash::flash_fwd_splitkv_kernel      28 launches/step   14.7 - 15.1 us each
flash::flash_fwd_splitkv_combine_kernel 28 launches/step    6.8 - 6.9 us each

28 = one per layer. the split and the combine, exactly as described above.

attention cost per step = 28 x (15.0 + 6.85) us = 612 us = 0.61 ms
whole decode step      = 12.56 ms
attention share         = 4.9%
```

The number that ties Days 9-11 to Days 5-8. Attention is **5%** of a batch-1 decode step. The other 95% is streaming 3.09 GB of weights. So a perfect attention kernel would buy you 5% at best. This is the honest, quantified reason FlashAttention is not the answer to decode latency, and it is why the entire Week 1 roofline analysis pointed at weight bandwidth instead.

Method note: the trace also holds 28 longer split-KV launches (up to 122.9 us) from the first step, which includes prefill. Those are excluded above; the figures are steady-state decode only.

11.4 When each kernel is the right one

Regime	Query length	Bottleneck	Right kernel
Prefill / training	N, large	Writing the N-by-N score matrix to HBM	FlashAttention 1 / 2
Decode, batch 1	1	Reading the KV cache, and idle SMs	Flash-decoding (split-KV)
Decode, large batch	1 per sequence, many sequences	Weight bandwidth, amortised across the batch	Batched attention; splitting matters less because batch already fills the SMs

Sources: [Dao et al.](#), [Flash-Decoding for long-context inference](#) · [FlashAttention-2](#) · [PagedAttention](#)

Day 12 · Continuous batching, chunked prefill, and the vocabulary

12.1 What static batching wastes

Group eight requests, run them together, return when all eight finish. The one that generates 500 tokens holds the batch slot of the one that finished at 20. Those seven slots compute padding for the rest of the run. On real traffic, where output lengths vary by an order of magnitude, most of the batch is idle most of the time.

12.2 Continuous batching (Orca)

Orca's contribution is **iteration-level scheduling**. The scheduler runs between every decode step, not between batches:

1. A sequence that emitted its stop token leaves the batch immediately, at the end of that step.
2. A queued request takes the free slot at the start of the next step.
3. No request waits for an unrelated request to finish.

The second half of the paper is **selective batching**: the linear layers batch cleanly because every sequence contributes one token, but attention cannot, because each sequence has a different context length. So the runtime batches the linear layers and runs attention per sequence. That asymmetry is the same one from Day 1: the MLP treats tokens independently, attention does not.

12.3 Chunked prefill (Sarathi)

Prefill and decode compete for the same GPU, and they are the two regimes from Day 5. A 2000-token prefill is compute-bound and occupies the device for a long single step. Every decode in flight stalls behind it, so their TPOT spikes. Users see a stutter caused by someone else's prompt.

Sarathi splits the prefill into fixed-size chunks and schedules one chunk per step, filling the rest of that step with decode tokens from running requests. The prefill takes marginally longer in total, but no single step is long, so TPOT stays flat. The trade is stated plainly: **slightly worse TTFT for the new request, much better TPOT for everyone already running**.

12.4 The vocabulary, precisely

Term	Definition	Your measured value, batch 1
TTFT	Time to first token: prefill plus the first sample	112 ms
TPOT / ITL	Time per output token after the first	11.28 ms
End-to-end	$TTFT + TPOT \times (out_tokens - 1)$	$112 + 11.28 \times 255 = 2989$ ms
p50 / p95 / p99	Percentiles across requests. Tail latency, not average	The number an SLO is written against
Throughput	Total output tokens per second across all requests	88.6 tok/s
Goodput	Throughput counting <i>only</i> requests that met their SLO	The metric that stops you gaming throughput with huge batches

12.5 Your sweep has two different ceilings, measured separately

This is the part of the repo that is genuinely uncommon. The same sweep was run twice with different KV pool sizes, and the throughput flattens for a different reason each time.

EXAMPLE 12-A: THE COMPUTE CEILING (REALISTIC POOL, 0.9 UTILISATION)

pool: 14,836 blocks x 16 tokens = 237,376 tokens = 309 concurrent sequences
nothing ever queues in this configuration.

batch	TPOT ms	tok/s	read
1	11.28	88.6	memory-bound, tensor cores idle
16	17.13	934	nearly free scaling
64	23.73	2697	<- knee: 30x throughput for 2.1x latency
128	44.30	2889	+7% throughput, +87% latency
256	86.29	2967	+10% throughput, +264% latency

ceiling ~2700-2970 tok/s. cause: the batched GEMM saturates the tensor cores.
this is the AI(B) = B crossing from Day 5.

EXAMPLE 12-B: THE KV ADMISSION CEILING (CONSTRAINED POOL, 0.4 UTILISATION)

pool: 1,224 blocks x 16 tokens = 19,584 tokens = 25 concurrent sequences

batch	TPOT ms	tok/s	queued?	
16	17.02	940	False	all 16 fit in the pool
32	33.13	966	TRUE	32 > 25, so 7+ sequences wait
48	43.82	1095	TRUE	
64	67.38	950	TRUE	throughput has stopped rising entirely

ceiling ~950-1095 tok/s, less than HALF the compute ceiling.
cause: the pool holds 25 sequences. the flag in the CSV flips at exactly the batch size that exceeds it.

How to tell this in an interview. "I measured two plateaus on the same GPU and the same model. With a full KV pool, throughput flattens at about 2700 tok/s because the tensor cores saturate, and the arithmetic-intensity model predicts that crossing. With the pool cut to 40% of memory, it flattens at about 1000 tok/s instead, and the engine's own queue flag turns on at exactly batch 32, which is the first batch larger than the pool's 25-sequence capacity. Same curve shape, two completely different causes, and you can only tell them apart by instrumenting admission rather than memory, because a paged engine's VRAM is flat either way."

12.6 TTFT is where batching costs you, and it is visible

EXAMPLE 12-C: TTFT AGAINST BATCH SIZE, REALISTIC POOL

batch	TTFT s	per-prompt prefill
1	0.112	
2	0.218	
8	0.727	
16	1.405	
32	2.774	
48	4.145	37x the batch-1 TTFT

TTFT is almost exactly linear in batch size, because prefill is compute-bound (Day 5) so the prompts cannot overlap. they serialise.

That linear curve is the entire motivation for chunked prefill in one plot. The 48th user waits 4.1 seconds for a first token because 47 other prompts are ahead of them, and no amount of KV cache fixes it, because the limit is the compute ceiling and not memory.

12.7 Exit bar for Days 9-12

1. Write the online-softmax recurrence from a blank page and work it on a four-element example that matches the direct computation.

2. State what FlashAttention changes (HBM traffic) and what it does not (FLOPs), and give the quadratic traffic figure for a 4096-token prefill on this model.
3. Give both reasons FlashAttention does nothing at batch-1 decode, then describe split-KV and its combine step.
4. Quote attention as 5% of your decode step, and say what that implies about where to optimise.
5. Describe iteration-level scheduling and why attention cannot be batched the way linear layers can.
6. Name your two plateaus, their values, and the distinct mechanism behind each.

12.8 Self-test

Q1. Softmax needs the whole row for its denominator. How can attention be tiled?

Keep a running max `m`, a running sum `1`, and a running output `0`. When a new tile has a larger max, multiply `1` and `0` by `exp(m_old - m_new)` and continue. The result is exact, not approximate: on scores `[1,3,2,5]` with values `[10,20,30,40]` the two-tile recurrence and the direct computation both give 36.8812.

Q2. Does FlashAttention reduce the complexity of attention?

No. Compute stays $O(N^2d)$ and the backward pass recomputes scores rather than storing them, so it does more arithmetic. What drops from $O(N^2)$ to $O(N)$ is HBM traffic. It is an IO-aware algorithm, and it wins because attention was memory-bound, not compute-bound.

Q3. You added FlashAttention and batch-1 decode did not get faster. Why?

Because at query length 1 the score matrix is a 1-by-n row, about 1 KiB, so there is nothing to keep out of HBM, and FA's parallelisation over query blocks yields 12 blocks for 28 SMs. The real cost is reading the KV cache and the weights. In my trace, attention is 0.61 ms of a 12.56 ms step, under 5%. The fix for the parallelism half is flash-decoding, which splits the KV sequence across SMs and merges the partials with the same online-softmax rescale. My vLLM trace shows exactly that: 28 split-KV kernels and 28 combine kernels per step, one pair per layer.

Q4. How would you pick a batch size for a production SLO?

Measure the throughput-latency Pareto and find the knee, then walk back to the largest batch that still meets the TPOT budget. On my sweep the knee is 64: 30x throughput for 2.1x latency up to that point, then 10% more throughput for 264% more latency out to 256. I would also check which ceiling I am hitting, because they need different fixes. My compute ceiling was 2700 tok/s and my KV admission ceiling, with a deliberately small pool, was 1000 tok/s, and only the second one is solved by buying memory.

Sources: Yu et al., Orca (OSDI 22) · Agrawal et al., Sarathi (chunked prefill) · Sarathi-Serve · Anyscale, Continuous batching · Databricks, LLM Inference Performance Engineering

Repo sources for every measured number above: `results/trace_vllm-graph.json.gz`, `results/profile_summary.csv`, `results/vllm_batch_sweep.csv`, `docs/profiling.md`, `docs/batching-results.md`. Batch-sweep throughput is derived as `batch / TPOT`. Where a CSV and a prose doc disagree, the CSV is used.